

Issues

HelixCode — Open Issues Tracker

Per Constitution §11.4.15 (Item-status tracking) + §11.4.16 (Item-type tracking) + §11.4.19 (Fixed-document column-alignment) + CONST-057 (Type-aware closure vocabulary) + CONST-058 (Reopened-source attribution).

Authoritative resumption ledger: docs/CONTINUATION.md (CONST-044). This file complements it with item-level granularity for currently-open work.

Status vocabulary (closed set): Queued | In progress | Ready for testing | In testing | Reopened | Fixed/Implemented/Completed (→ Fixed.md)

Type vocabulary (closed set): Bug | Feature | Task

Prefix convention

Round 189 (2026-05-19) introduced per-project / per-submodule ID prefixes replacing the legacy ISSUE-NNN flat namespace. New items **MUST** use the prefix matching their scope; cross-cutting items affecting two or more submodules (or any meta-repo concern) live under HXC. Numeric portion is per-prefix (each prefix starts at 001). Legacy ISSUE-NNN IDs renamed forward-only per CONST-043; historical close-out narratives in docs/CONTINUATION.md preserve original IDs verbatim, and docs/Fixed.md annotates each closure with (ex-ISSUE-NNN) for git-history traceability.

Prefix	Scope	Source
HXC	HelixCode root project (project-wide, multi-submodule, governance, infrastructure)	this repo
HXA	HelixAgent submodule	submodules/helix_agent (when present) / helix_agent / tree

Prefix	Scope	Source
HXM	HelixMemory submodule	submodules/ helix_memory
HXL	HelixLLM submodule	submodules/helix_llm
HXQ	HelixQA submodule	submodules/helix_qa (helix_qa/)
HXS	HelixSpecifier submodule	submodules/ helix_specifier
HXO	HelixOrchestrator (= LLMOrchestrator) submodule	submodules/ llm_orchestrator
HXV	HelixVerifier (= LLMsVerifier) submodule	submodules/ llms_verifier
HXD	HelixDocProcessor (= DocProcessor) submodule	submodules/ doc_processor
HXI	HelixI18n (when added)	tba
PLN	Planning submodule (vasic-digital)	submodules/planning
VEN	VisionEngine submodule (HelixDevelopment)	submodules/ vision_engine
SLF	SelfImprove submodule (vasic-digital)	submodules/ self_improve
STO	Storage submodule (vasic-digital)	submodules/storage
OPS	LLMOps submodule (vasic-digital)	submodules/llm_ops
VDB	VectorDB submodule (vasic-digital)	submodules/vector_db
OBS	Observability submodule (vasic-digital)	submodules/ observability
MCP		

Prefix	Scope	Source
	MCP_Module submodule (vasic-digital)	submodules/ mcp_module
MSG	Messaging submodule (vasic-digital)	submodules/messaging
MDW	Middleware submodule (vasic-digital)	submodules/ middleware
PLG	Plugins submodule (vasic-digital)	submodules/plugins
STR	Streaming submodule (vasic-digital)	submodules/streaming
WAT	Watcher submodule (vasic-digital)	submodules/watcher
CNV	conversation submodule (vasic-digital)	submodules/ conversation
AUT	Auth submodule (vasic-digital)	submodules/auth
LZY	Lazy submodule (vasic-digital)	submodules/lazy
ATP	AutoTemp submodule (vasic-digital)	submodules/auto_temp
PLI	PliniusCommon submodule (vasic-digital)	submodules/ plinius_common
CHL	challenges submodule (vasic-digital)	challenges/
CNT	containers submodule	containers/
SEC	security submodule	security/
PAN	panoptic submodule	panoptic/

For submodules not listed above, default to the first 3 letters of the submodule name, uppercase (e.g. Watcher → WAT). Document the new prefix in this table on first use.

Legacy → new mapping (round 189)

Old ID	New ID	Scope rationale
ISSUE-001	VEN-001	VisionEngine helix-gitlab URL
ISSUE-002	VEN-002	VisionEngine vasic-digital-github fork divergent
ISSUE-003	HXL-001	HelixLLM analysis_test.go hardcoded path
ISSUE-004	HXL-002	HelixLLM TOON WriteTOON 500
ISSUE-005	HXC-001	CONST-052 rename programme (project-wide)
ISSUE-006	HXC-002	Round-74 residual LOGIC FAILs (multi-submodule cross-cutting)
ISSUE-007	HXC-003	CONST-046 migration backlog (project-wide)
ISSUE-008	HXQ-001	helix_qa TestPerformance flake
ISSUE-009	HXA-001	helix_agent handler tests (4)
ISSUE-010	HXA-002	helix_agent debate API drift
ISSUE-011	HXA-003	venice CONST-037 (in helix_agent)

HXC-154 — TestStartQASession_Success asserts on a session field a background goroutine is concurrently changing

Status: Queued **Type:** Bug **Severity:** Medium **Created-By:** Claude

A QA-session test checks that a freshly started session reports the status ‘pending’, but the value it reads is one that a background worker is racing to change to ‘running’ at that very moment. The server hands the JSON encoder the same live session object the worker mutates, so whether the response says ‘pending’ or ‘running’ depends purely on which goroutine wins, and under load or with the race detector enabled the worker can win. The result is a test that usually passes and occasionally fails for reasons unrelated to any real product defect, which erodes trust in the suite and trains readers to dismiss red runs. It was observed failing once during unrelated work, then passed eleven consecutive full-package race runs afterwards, so it is genuinely rare rather than broken outright. The underlying product behaviour is not necessarily wrong — a caller may legitimately see either state — so the likely correct outcome is to make the

assertion accept either value or to observe the status through a synchronised read instead. Done means the test no longer depends on goroutine scheduling, proven by repeated runs under the race detector, with the reproduction captured first.

HXC-158 — No test builds the harmony_os or aurora_os screens, so their widget-threading fixes rest on code review rather than captured runtime proof

Status: Queued **Type:** Bug **Severity:** Medium **Created-By:** Claude

The Harmony OS and Aurora OS front-ends were just corrected so that background workers no longer modify on-screen elements directly. That correction is, however, only partly backed by evidence. No test in either package ever constructs the application's screens, so the background workers that paint them are never started while the tests run, and the race detector therefore never gets the opportunity to observe the very code paths that were changed. Concretely, running the Aurora OS package under the race detector reported zero races both before and after the fix — not because the defect was absent, but because nothing under test reaches it; the corresponding Harmony OS failure that was reproduced and fixed came from a different component, the background system monitor, which tests do start. The practical risk is that a future edit could silently reintroduce a direct on-screen write from a background worker in either file and every test would stay green, exactly the situation the anti-bluff policy exists to prevent, since a green suite would then be certifying behaviour nobody has actually exercised. Closing this requires a test that builds the tabs and drives those workers — the chat worker, the provider-health poller and the dashboard and resource timers — with the race detector on, so the fix is proven by observation rather than by inspection, together with a deliberately-broken variant proving the new test genuinely fails when the direct write is put back. Acceptance: a test in each package starts the previously-unexercised background workers against real widgets, passes with zero races over repeated runs, and demonstrably fails when the dispatch helper is removed from any one of those sites. Until that exists, the fixes for those specific sites should be described as review-justified, not runtime-proven.

HXC-159 — Fully incorporate the HelixSkills System into HelixCode + HelixAgent (all power-features, SpecKit-driven)

Status: Queued **Type:** Task **Severity:** High **Created-By:** Operator

Reported-Via: §11.4.202 reporting directive task on 2026-07-28T19:26:22Z

Reported-By: Operator

What (the report, verbatim): Feature description (verbatim): Fully incorporate the HelixSkills System (git@github.com:HelixDevelopment/skills.git) into BOTH HelixCode and HelixAgent, covering ALL of its power-features with nothing left out.

VERBATIM OPERATOR REQUIREMENTS (2026-07-29): “Fully incorporate into HelixCode and HelixAgent the HelixSkills System for all work with Skills: git@github.com:HelixDevelopment/skills.git , we MUST fully incorporate all HelixSkills power-features fully without nothing leftout! Full exhaustive implementation plan divided into phases, tasks and subtasks with exhaustive amount of data with all details MUST BE prepared! Full coverage with ll supported test types, challenges and HelixQA test banks (suites) is mandatory! Extend and update all existing documentation, user manuals, guides, FAQs, create stunning illustrations, graphs and diagrams and incorporate them in all materials! Any gaps, shortcomings, weak spots, danger zones, issues, or any inconsistencies MUST BE detected in advance during the planning and implementation process and properly tackled with comprehensive rock-solid solutions implementations, risk-free and safe! Track all progress through the constinuation docs, memory, knowledge and remebering mechanisms we have! Use GitHub SpecKit for all phases of incorporating with bridge to Superpowers used for the implementation and subagents driven development mechanism!”

PRE-VERIFIED ENVIRONMENT FACTS (established 2026-07-29, not assumed — §11.4.6): - GitHub SpecKit IS installed at /home/milos/.local/bin/specify. It is NOT yet initialized in this repo (no .specify/ and no specs/ directory present).

Initialization is therefore an explicit early task, not an assumption. -

git@github.com:HelixDevelopment/skills.git IS reachable via SSH. git ls-remote --heads returns branches including feature/catalog-docs, feature/deep-research, feature/testing-infra. The default branch and full branch/tag inventory MUST be re-derived at research time rather than inferred from this partial listing. - The repository is NOT currently a submodule of this project. .gitmodules contains no HelixDevelopment/skills entry; the only ‘skills’-matching entry is cli_agents/codex-skills -> vasic-digital/caf-codex-skills.git, which is a DIFFERENT repository and must not be confused with it. - Both consumers exist in-tree: the inner Go module helix_code/ (module dev.helix.code) and the submodule submodules/helix_agent (module dev.helix.agent).

MANDATORY DELIVERY CONSTRAINTS: - SpecKit-first: use GitHub SpecKit for ALL phases of the incorporation, bridged to Superpowers for implementation, executed subagent-driven (§11.4.70 / §11.4.20). - Exhaustive plan structured as phases -> tasks -> subtasks, with an exhaustive level of detail (down to lines-of-code and micro-POCs where the design is load-bearing). - Complete test coverage

per §11.4.169: unit, integration, e2e, full-automation, Challenges (vasic-digital/challenges), HelixQA banks (HelixDevelopment/helix_qa) with autonomous sessions, DDoS, security, stress + chaos, concurrency/atomicity, race/deadlock, memory, benchmarking. Every PASS carries captured evidence (§11.4.5 / §11.4.69); every guard ships a paired §1.1 mutation. - Documentation: extend AND update every existing doc, user manual, guide and FAQ — not only new ones. Produce illustrations, graphs and diagrams and incorporate them into all materials. Four-format export discipline per §11.4.65 / §11.4.153. Every doc reachable from the main README per §11.4.212. - Risk work is a PLANNING deliverable, not a post-hoc note: every gap, shortcoming, weak spot, danger zone, issue and inconsistency MUST be detected DURING planning and each one paired with a comprehensive, rock-solid, risk-free and safe solution (§11.4.102 systematic-debugging over all gathered data). - Reuse before rewrite (§11.4.74 / §11.4.28): survey the vasic-digital and HelixDevelopment catalogues on GitHub AND GitLab first; extend owned submodules in place rather than duplicating, keeping them project-agnostic and decoupled. HelixSkills itself must be incorporated as a properly-laid-out dependency per §11.4.28(C) / CONST-051(C) — root-level or submodules/, never a nested own-org chain. - Cross-consumer parity: HelixCode and HelixAgent must BOTH receive the full feature set; a capability landing in one and not the other is an incompleteness to be tracked, not silently accepted. - Progress tracked through the continuation docs, the memory/knowledge/remembering mechanisms, and the workable-items SSoT (§12.10 / §11.4.131 / §11.4.93 / §11.4.95).

ACCEPTANCE CRITERIA: 1. A SpecKit specification exists and is initialized in-repo, covering every HelixSkills power-feature, with an explicit feature inventory proven complete against the upstream repository (an enumerated list, not a sample — §11.4.118). 2. A phase/task/subtask implementation plan exists at the stated depth, with each risk item paired to a concrete mitigation. 3. Every planned capability is wired and end-user-reachable in BOTH consumers, proven by §11.4.108 runtime signatures on a clean target — not merely compiled. 4. The full §11.4.169 test-type matrix is green with captured evidence, Challenges and HelixQA banks included. 5. All documentation is updated and exported, with the produced diagrams/illustrations incorporated, and reachable from README. 6. Independent code review reaches a zero-finding GO per §11.4.125 / §11.4.134 / §11.4.142. 7. No capability is claimed without runtime evidence — the anti-bluff covenant governs every closure (§11.4 / §11.4.1 / §11.4.123).

§11.4.213 FEATURE research-and-planning WORK PROGRAM (scheduled, not yet executed):

1. Deep web-research series (articles / guides / scientific papers / open-source projects+codebases / real-world examples) on the best way to incorporate this feature into the project (§11.4.8 / §11.4.99 / §11.4.150).

2. Systematic-debug (§11.4.102) of ALL data obtained to enumerate EVERY weak spot / gap / danger-zone / inconsistency / imperfection, and design a risk-free, rock-solid solution for EACH one found.
3. The design MUST ALWAYS be enterprise-grade, bleeding-edge, and innovative – never less.
4. MANDATORY exhaustive documentation: many technical documents, an implementation plan down to lines-of-code + micro-proof-of-concepts, diagrams / schemes / graphs, illustrations, SQL definitions, templates, and every other relevant material (§11.4.65 / §11.4.73).
5. Investigate + plan REUSE of existing vasic-digital + HelixDevelopment submodules/components BEFORE proposing a rewrite; extend them freely where genuinely needed while keeping them fully decoupled / project-not-aware / reusable (§11.4.28 / §11.4.74 / §11.4.177).
6. Plan from the TESTING point of view from day one: every supported test type, the Challenges submodule, and full HelixQA bank coverage (§11.4.27 / §11.4.169).
7. Divide the work into phases / tasks / subtasks, fine-grained and nano-detailed enough to drive integration / implementation / wiring / testing / scaling directly.
8. Plan explicitly for enterprise scalability and maximal performance.
9. When the feature has a UI/UX surface, produce full wireframes / diagrams / design files (Figma / PSD / PDF, or whatever the project mandates) authored via OpenDesign (§11.4.162 / §11.4.190).
10. Plan full CodeGraph integration with regular / real-time index synchronisation (§11.4.78 / §11.4.79 / §11.4.80).
11. Create fully-detailed follow-on workable items (every detail + reference + attachment) synced in REAL TIME to the SQLite single-source-of-truth (§11.4.93 / §11.4.95) + every derived workable-items document + every related project doc/component + every connected external work-tracking system (§11.4.148 D5) – honestly SKIPPING any absent tracker with a machine-readable reason (credentials_absent / tracker_client_absent, §11.4.10) rather than EVER faking a push.

Research-doc destination (to be populated when this item is worked): docs/
research/

fully_incorporate_the_helixskills_system_into_helixcode_heli_20260728T192622Z_2557683/

Execution model (§11.4.213 clauses 1-2): this item is SCHEDULED, not yet executed. The multi-day research/planning/documentation work above is performed LATER by the project's standing autonomous loop (§11.4.87 / §11.4.94 / §11.4.97 / §11.4.103 / §11.4.126) when it claims this item, and MUST be driven to a genuinely COMPLETED-and-wired or explicitly evidence-backed CLOSED terminal state under §11.4.197 – this item MUST NOT be left sitting un-wired in the backlog.

Affected scope / file-scope manifest: Feature research/planning – destination: docs/research/fully_incorporate_the_helixskills_system_into_helixcode_heli_20260728T192622Z_2557683/

Reproduction / context: UNKNOWN: not stated in the report — a reproduction MUST be established before any fix (§11.4.102 / §11.4.146 / §11.4.199)

Acceptance criteria: The full research-doc tree exists under docs/research/fully_incorporate_the_helixskills_system_into_helixcode_heli_20260728T192622Z_2557683/, every enumerated weak-spot/gap/danger-zone carries a designed risk-free solution, an implementation plan down to lines-of-code exists, the planned follow-on workable items are created, and the whole effort is validated per §11.4.197 (never left un-wired).

=== ADDENDUM — BINDING OPERATOR REQUIREMENT (2026-07-29, added after scheduling) ===

VERBATIM: “Make sure that SpecKit and Superpowers are used with Bridge extension and with all extensions we have created derived from constitution Submodule!”

This is not optional colour on the SpecKit mandate — it names specific existing assets that this work MUST compose with rather than duplicate (§11.4.74 extend-don’t-reimplement).

THE BRIDGE EXTENSION (verified to exist, 2026-07-29): constitution/docs/research/extensions/speckit_superpowers/implementation/ — the “Helix Constitution-Powered SpecKit-Superpowers Bridge”. Eight documents, each in .md/.html/.pdf/.docx: README, IMPLEMENTATION_PLAN, NANO_TASK_ENGINE, EXTENSION_DEVELOPMENT, TDD_INTEGRATION, CONSTITUTION_INTEGRATION, SECURITY, APPENDIX.

It defines a SEVEN-LAYER architecture: 1 Developer Workstation 2 Spec-Kit Core (governance) 3 SuperSpec bridge (orchestration — github.com/WangX0111/superspec) 4 SuperB extension (discipline enforcement — speckit-community.github.io/extensions/superb) 5 SuperBridge MCP (execution) 6 Helix LLM (inference — github.com/HelixDevelopment/helix_llm) 7 Distributed host cluster (llama.cpp RPC)

CONSTITUTION-DERIVED EXTENSIONS THAT MUST BE IN SCOPE (inherited BY REFERENCE per §11.4.228 — never copied locally, a copy diverges silently): constitution/skills/ (7) action-prefix-system, media-validator, multitrack, reporting-workable-items, scheduled-work-queue, session-sync, workable-item-lifecycle constitution/mcp/ (2) media-validator-mcp.json, scheduled-work-mcp.json constitution/plugins/ (2) helix, scheduled-work constitution/actions/ registry.yaml (§11.4.140 action system), subagent_tiering.yaml constitution/scripts/hooks/ (7

live) `action_prefix_expand.sh`, `credential_scan_lib.sh`, `guard-branch-consistency.sh`, `guard-forbidden-commands.sh`, `guard-track-branch-label.sh`, `guard-work-track-binding.sh`, `post-merge`

ALREADY PRESENT LOCALLY — do not re-create: `.claude/skills/` 10 registered speckit-* skills (analyze, checklist, clarify, constitution, converge, implement, plan, specify, tasks, taskstoissues) `.specify/ workflows/speckit/workflow.yml`, `integrations/{claude,speckit}.manifest.json`, `scripts/bash/`, `templates/`, `memory/`

ADDED ACCEPTANCE CRITERIA: 8. Every constitution-derived extension above has an explicit disposition — reused, extended, or orthogonal-with-reason. Silence is not an answer (§11.4.118). 9. Each of the 7 Bridge layers is classified WIRED vs DESIGNED-ONLY with a captured probe. A design document existing is NOT evidence a layer is installed — that is precisely the §11.4.108 SOURCE-vs-RUNTIME gap, and reporting a documented layer as an available one would be a §11.4 bluff. 10. The skill-namespace collision risk is addressed with a designed mitigation: HelixSkills is itself a skills system, while §11.4.164's `post_update_hook.sh` already registers constitution skills into `.claude/skills/`. Two systems registering into one namespace needs defined precedence and clash handling, not discovery at runtime. 11. SuperSpec and SuperB are THIRD-PARTY, non-own-org dependencies (§11.4.74 vendor path) — their health, maintenance status and supply-chain exposure must be assessed, not assumed from the design doc referencing them.

HXC-160 — Governance manuals still name the superseded shell summary generators as the canonical tracker-summary generators

Status: Queued **Type:** Task **Created-By:** Claude

The project's agent manuals — `CONSTITUTION.md`, `CLAUDE.md`, `AGENTS.md`, `QWEN.md`, `GEMINI.md` and `CRUSH.md` at the repository root, their copies under `helix_code/` and `github_pages_website/`, and the cascaded copies inside the formatters, plugins and models submodules — still tell every reader that `docs/Issues_Summary.md` and `docs/Fixed_Summary.md` are produced by the shell scripts `scripts/generate_issues_summary.sh` and `scripts/generate_fixed_summary.sh`. That has not been true since the workable-items SQLite database became the single source of truth: both summaries are now generated from that database by the constitution submodule's Go exporter, while the shell scripts derive them from the Markdown trackers instead, which silently drops every item recorded as a heading section rather than a table row. The practical harm is that an agent doing exactly what the manual says destroys tracked state: on 2026-07-29 a hand invocation rewrote the closed-item tally from 344 to 188, corrupted every per-type count, and replaced the full per-item table with aggregate counts only, before the sync gate caught it. The scripts themselves

have now been changed to refuse to run, so the immediate danger is closed, but the manuals still point at them, so every new agent is still given the wrong instruction and discovers the refusal only by tripping over it. Fixing this means updating the CONST-057 and section 11.4.91 anchor text in every carrier so it names the database exporter as the generator, which spans roughly twenty files including copies inside submodules that other repositories own, and must keep all five carriers in lockstep per section 11.4.157. Who benefits: any agent or engineer who reads a manual to learn how to regenerate a tracker summary, and everyone relying on those summaries to see true project state. Reproduce by opening any listed manual and searching for `generate_fixed_summary.sh` — the surrounding sentence presents it as the current generator with no mention of supersession. Acceptance: every carrier names the DB exporter, no carrier presents the shell scripts as current, the five root carriers stay in lockstep, and a check exists that would fail if a manual reintroduced the stale naming.

HXC-161 — Fixed.md H2-section-to-pipe-row parity gate fails on 58 items because the DB emits each item in only one representation

Status: Queued **Type:** Bug **Created-By:** Claude

The guard at `scripts/gates/fixed_h2_pipe_row_parity_gate.sh` checks that every closed item written in `docs/Fixed.md` as a detail section also appears as a row in that file's summary table. It currently reports 58 failures, and it was already failing before this batch — the same failures are reproducible against the last committed version of the file, so this is a standing red gate rather than something a recent change broke. The cause is a change in how the file is produced. The tracker is now generated from the `workable-items` database, and the database records for each item which single surface form it takes: some items are written as table rows and others as detail sections, never both. The guard was written earlier, when the table was maintained by hand and every item was expected to appear in both places, so it is now asserting a relationship the data model deliberately no longer provides. Two outcomes are possible and they must not be guessed between: either the guard is stale and should be rewritten to assert what the generator actually guarantees, or the generator is genuinely dropping rows that ought to exist and the guard is correctly catching real data loss. Deciding which requires reading the exporter's representation handling and confirming against the database, so this is deliberately not resolved here — weakening or deleting a failing guard without that investigation is exactly the suppression the constitution forbids. Who benefits: anyone relying on this guard to catch items vanishing from the closed-items summary, which is the precise failure it was created to prevent after one item went missing twice over. Reproduce by running `bash scripts/gates/fixed_h2_pipe_row_parity_gate.sh` from the repository root and

observing 58 failures naming sections such as HXC-013 and HXC-119. Acceptance: the correct branch is chosen with captured evidence, the guard passes for the right reason rather than by being softened, its paired mutation still makes it fail when the invariant is broken, and if the generator was at fault the affected items are restored.

HXC-162 — Skill auto-trigger is dead in the interactive CLI: the dispatcher is built then thrown away

Status: Queued **Type:** Bug **Created-By:** Claude

When a user types a request in the HelixCode interactive command-line client, the feature that is supposed to notice the request matches a known skill and run it automatically never fires. The code that decides which skill applies is created correctly and then immediately discarded without ever being connected to anything, so it can never act on what the user types. To the user this looks like the skills feature simply does not exist in the CLI, even though the underlying machinery is present, correct and documented. This matters because skills are how the product is meant to turn a plain-language request into real work without the user learning any commands. Anyone reading the source would reasonably conclude the feature is implemented, which is why this went unnoticed. The fix is to connect the dispatcher to the place where user input is handled, and prove it with a test that types a matching request and observes the skill actually run. Reported at [cmd/cli/main.go:1060](https://cmd/cli/main.go#L1060) by SOURCE-layer inspection; still needs runtime confirmation per 11.4.108.

HXC-163 — None of the seven shared governance skills are registered, so none of them can ever run

Status: Queued **Type:** Bug **Created-By:** Claude

The project ships seven ready-made skills that come from the shared governance component and are meant to be available to every project that uses it. In practice none of the seven is registered with the running system, so not one of them can be triggered by a user or by an agent. The effect is that work the organisation already paid for and maintains sits unused, and each project quietly re-solves problems these skills already solve. Separately, one entry in the skills folder points at a storage location that only exists on a Mac, so on this Linux machine it is a broken link that resolves to nothing. These two problems compound: the catalogue looks populated from the outside while being empty in practice. Closing this means registering the seven skills through the supported mechanism, repairing or

removing the broken link, and adding a check that fails if a shipped skill is present on disk but unreachable at runtime.